

Classification Supervisée

KNN • SVM • Bayes Naïf

De l'intuition à la maîtrise : trier, séparer, prédire.
Trois algorithmes, trois philosophies, un même objectif.



1

KNN

Comparer aux voisins les plus proches pour décider



2

SVM

Tracer la meilleure frontière entre les classes



3

Bayes Naïf

Calculer la probabilité d'appartenance à une classe

Riadh Abdelfattah — SupCom 2026

— Sommaire

66 slides → 19 slides compactes en 5 actes

A **Rappels & Introduction**
Classification vs régression, vue d'ensemble

B **K Plus Proches Voisins (KNN)**
Intuition, étapes, choix de K

C **Support Vector Machine (SVM)**
Marge, soft margin, kernel trick

D **Classifieur Bayésien Naïf**
Règle de Bayes, vraisemblance, spam

E **Synthèse & Décision**
Comparaison, arbre de choix, récapitulatif

N1 Analogie & intuition

N2 Concepts clés

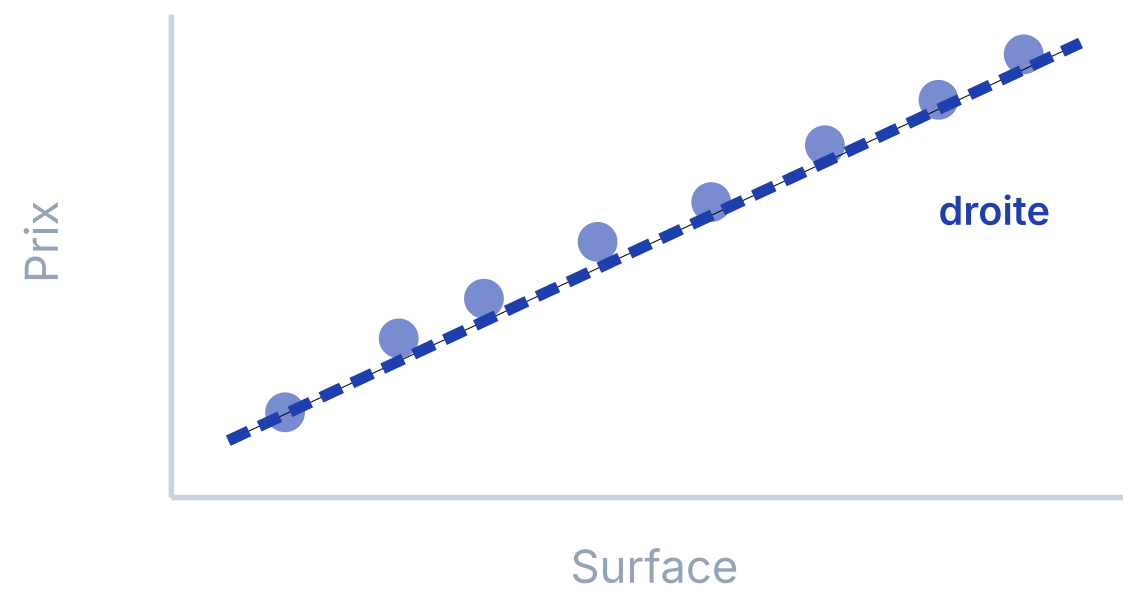
N3 Détails techniques

Classification vs Régression

N1 Régression = prédire un nombre

La sortie est une **valeur continue** : prix, température, note.

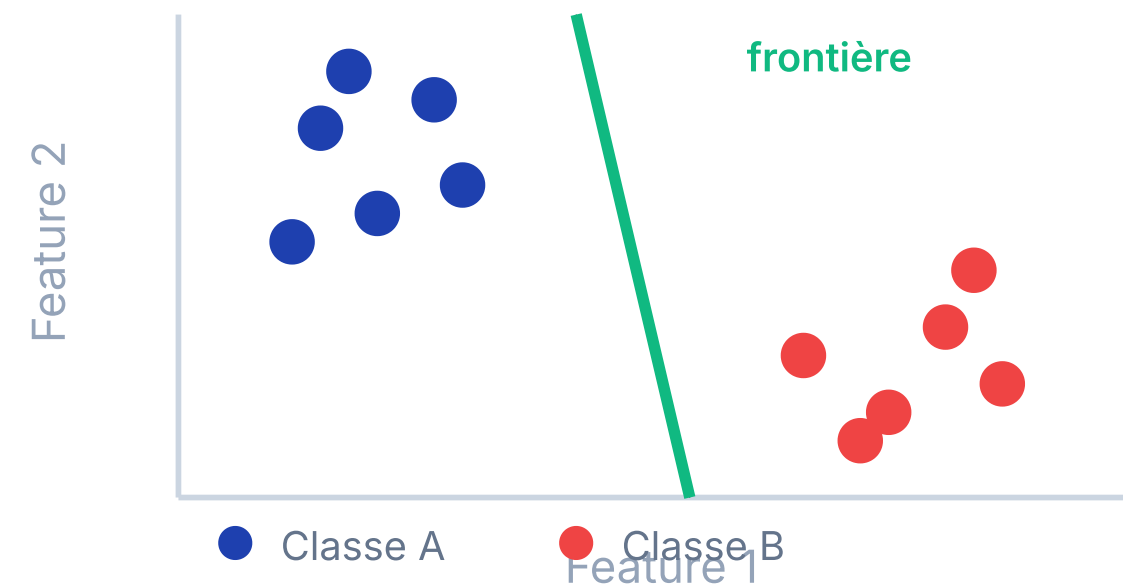
Exemple : surface → prix d'une maison (120 000, 250 000...)



N1 Classification = prédire une catégorie

La sortie est une **valeur discrète** : spam/non-spam, chat/chien, admis/refusé.

Exemple : symptômes → malade ou sain



La classification supervisée en une phrase

A partir d'un échantillon d'apprentissage $E = \{(x_i, y_i)\}$, on cherche une loi $f(x)$ qui prédit la **classe** d'une nouvelle observation. « Supervisée » parce que chaque exemple d'entraînement a une étiquette connue y_i .

Trois philosophies, un même objectif

KNN

Philosophie : « Dis-moi qui sont tes voisins, je te dirai qui tu es. »

Non-paramétrique

Memory-based

Garde *toutes* les données en mémoire.
Aucun modèle à entraîner.

SVM

Philosophie : « Trouver le meilleur mur entre deux camps. »

Paramétrique

Marge maximale

Cherche l'hyperplan qui maximise la *distance* entre les classes.

Bayes Naïf

Philosophie : « Mettre à jour ses croyances avec les preuves. »

Probabiliste

Indépendance

Calcule $P(\text{classe}|\text{données})$ via le théorème de Bayes.

Point commun

Les trois algorithmes apprennent une **frontière de décision** à partir de données étiquetées. Ils diffèrent dans la *façon* de tracer cette frontière.

K Plus Proches Voisins : l'intuition

N1 🏠 L'analogie du nouveau quartier

Tu déménages et tu veux savoir si ton **nouveau quartier est calme**.

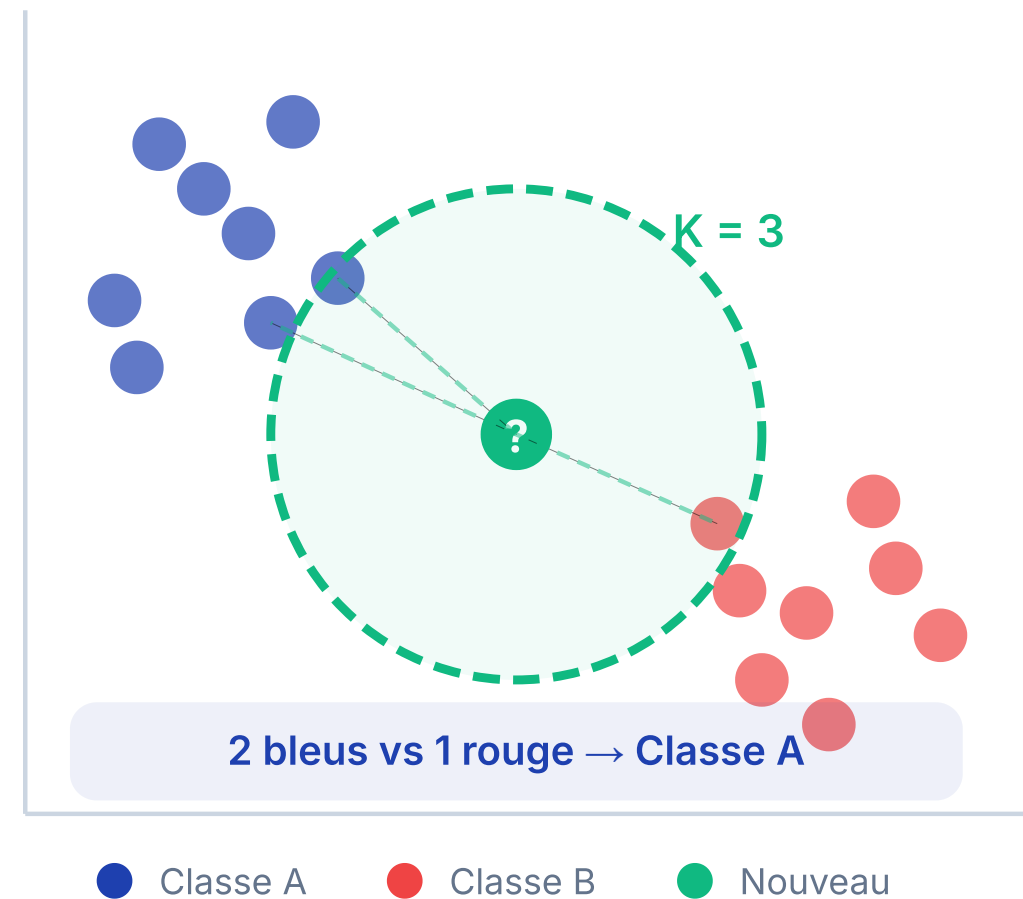
Tu demandes à tes **K voisins les plus proches**. Si la majorité dit « calme », tu classes le quartier comme calme.

💡 C'est exactement ce que fait KNN : voter par proximité.

📊 Pourquoi « non-paramétrique » ?

KNN n'apprend **aucun paramètre** ($\theta, w, b...$). Il stocke toutes les données et compare au moment de la prédiction. C'est un algorithme *memory-based* (à mémoire).

Principe visuel du KNN



Les 4 étapes de KNN

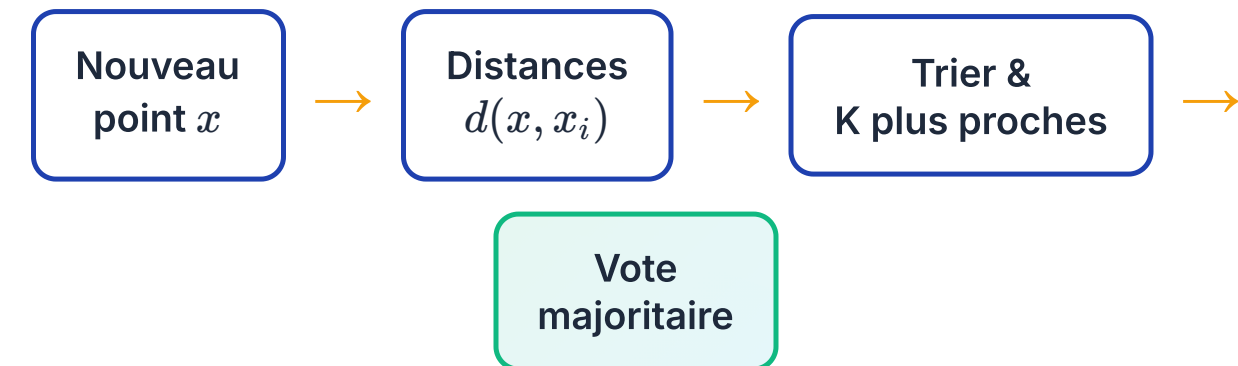
N2 L'algorithme pas à pas

- 1 Choisir **K** (nombre de voisins à consulter)
- 2 Calculer la **distance** entre le nouveau point et chaque point d'entraînement
- 3 Sélectionner les **K voisins** les plus proches
- 4 **Vote majoritaire** : la classe la plus fréquente parmi les K voisins l'emporte

Distance euclidienne

$$d(x, x') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$$

N1 Visualisation du processus



⚠ Attention

K est un hyperparamètre : il n'est pas appris par l'algorithme, il est choisi par vous.

K impair est préférable pour éviter les égalités de vote.

📈 Complexité

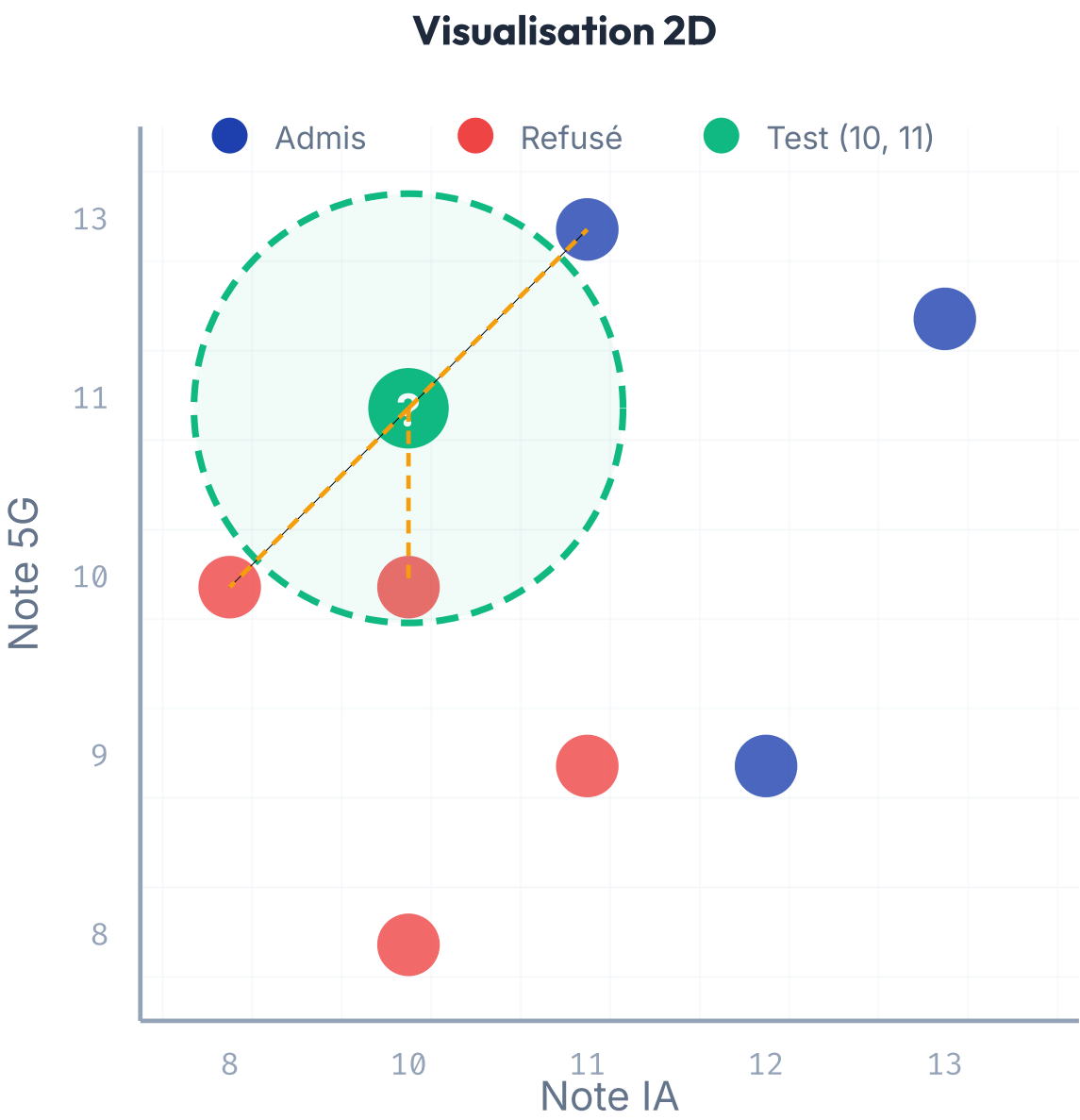
Prédiction : $O(n \times d)$ par requête (n = taille du dataset, d = dimensions). Lent sur de gros datasets, car il doit tout recalculer à chaque prédiction.

Exemple concret : résultats de thésards

N2 Données d'entraînement

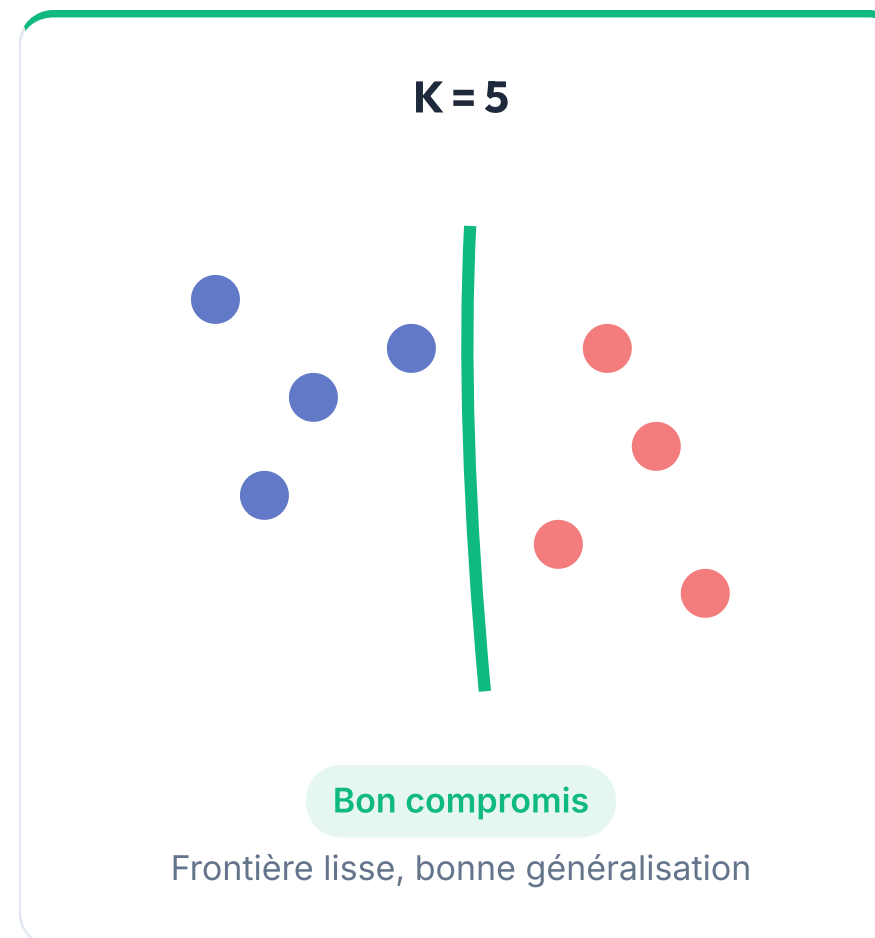
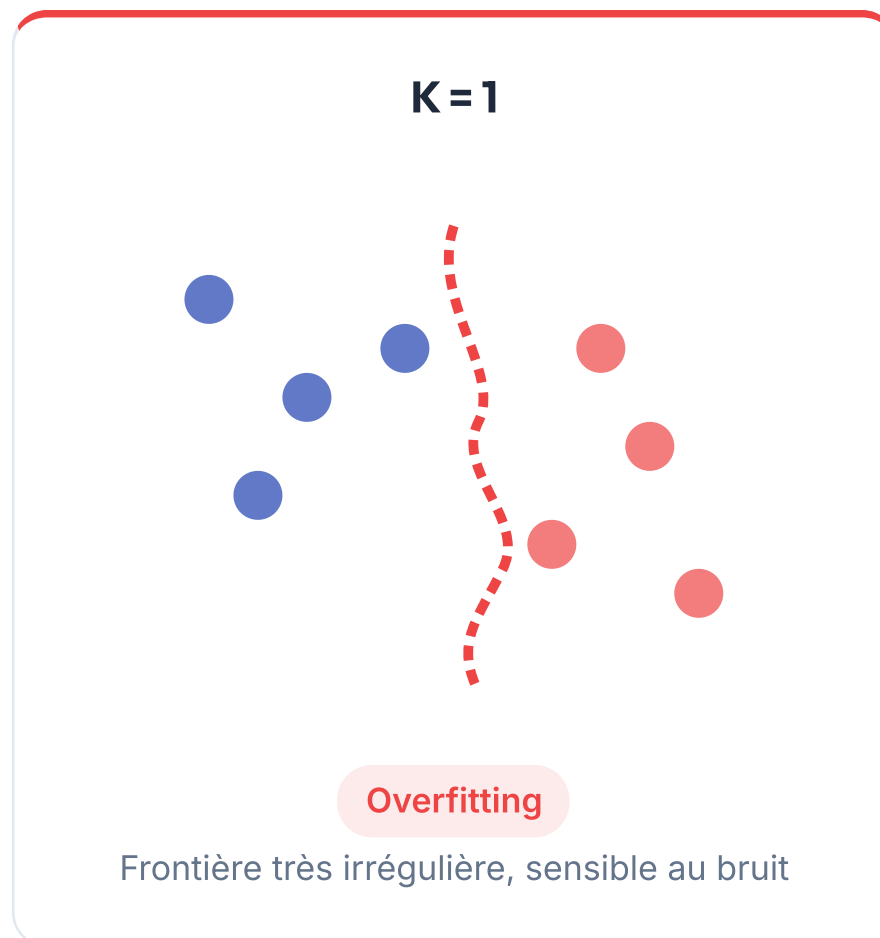
NOTE IA	NOTE 5G	RÉSULTAT
12	9	Admis
8	10	Refusé
10	10	Refusé
11	13	Admis
13	12	Admis
10	8	Refusé
11	9	Refusé

Question : Un thésard avec **10 en IA** et **11 en 5G** sera-t-il admis selon 3-NN ?



3 voisins les plus proches : (10,10) Refusé, (11,13) Admis, (8,10) Refusé
Vote : 2 Refusés vs 1 Admis → **Refusé**

L'impact du choix de K



⚖️ Le dilemme biais-variance

K petit = variance forte (sensible au bruit, overfitting)

K grand = biais fort (trop simple, underfitting)

Le bon K se trouve par **validation croisée**.

⚡ Normalisation obligatoire

Si une feature varie de 0 à 1 000 et l'autre de 0 à 1, la première **écrase** la seconde dans le calcul de distance. Il faut **normaliser** les données avant d'appliquer KNN.

Support Vector Machine : l'intuition

N1 L'analogie du terrain de foot

Deux équipes sur un terrain. Tu veux tracer **une ligne au sol** qui sépare les deux groupes.

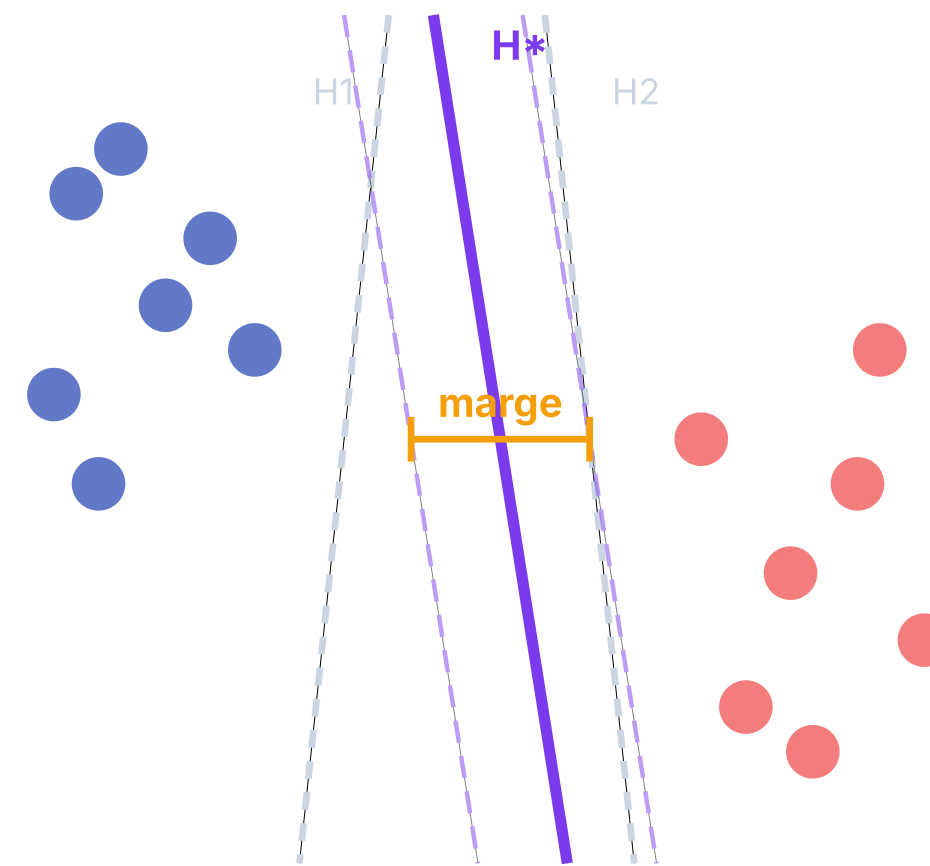
Pas *n'importe quelle* ligne : celle qui passe **le plus loin possible** des deux groupes. Plus la « rue » entre les deux est large, mieux c'est.

💡 SVM = maximiser la marge entre les classes.

Pourquoi maximiser la marge ?

Un hyperplan avec une **grande marge** généralise mieux sur de nouvelles données. C'est comme laisser un couloir de sécurité entre deux frontières.

Quel hyperplan choisir ?



H^* = hyperplan optimal (marge maximale)

Hyperplan, marge et vecteurs supports

N2 Les 3 concepts clés

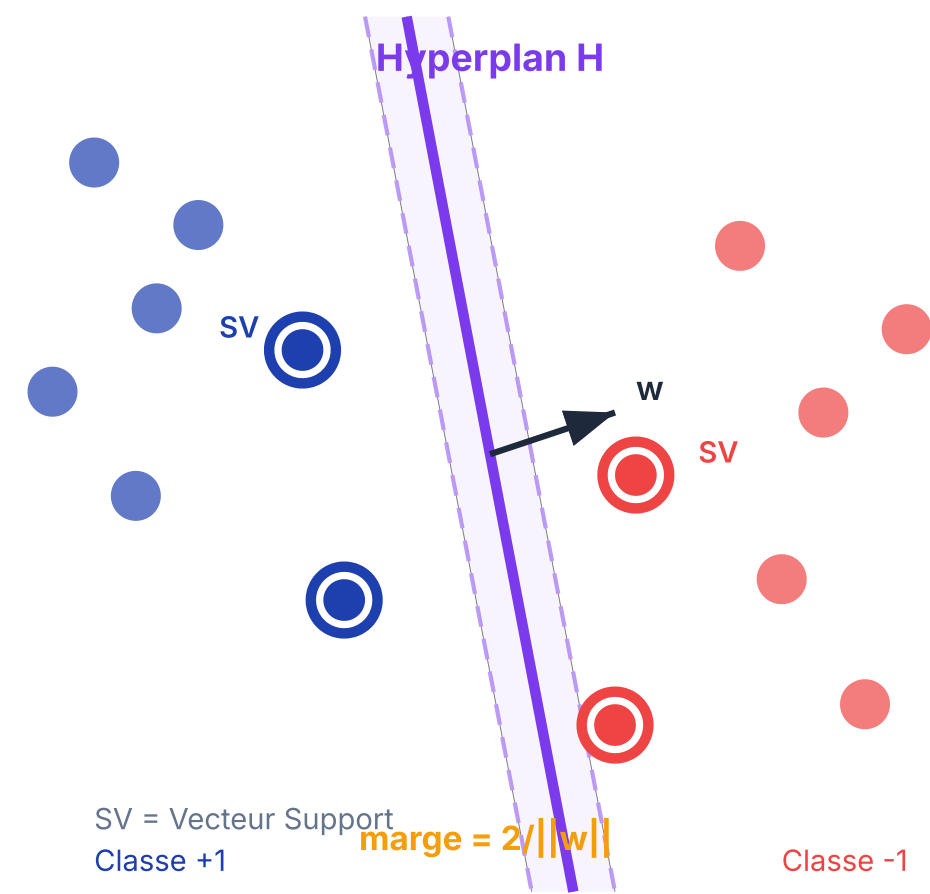
- **Hyperplan** : la frontière de décision. En 2D c'est une droite, en 3D un plan, en nD un hyperplan.
- **Marge** : la distance entre l'hyperplan et les points les plus proches de chaque classe.
- **Vecteurs supports** : les points qui « touchent » la marge. Ce sont eux qui *définissent* l'hyperplan.

Fonction de décision

$$f(x) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

$\mathbf{w}$ = vecteur normal à l'hyperplan, b = biais

Anatomie d'un SVM



Soft Margin : quand la séparation n'est pas parfaite

N1 L'analogie du vigile

Un vigile à l'entrée d'une soirée sépare les « invités » des « non-invités ». Mais parfois, il **tolère quelques erreurs** (un invité un peu douteux, un non-invité qui ressemble à un VIP).

Le paramètre **C** contrôle la tolérance du vigile :

C grand

Peu tolérant : peu d'erreurs, marge étroite

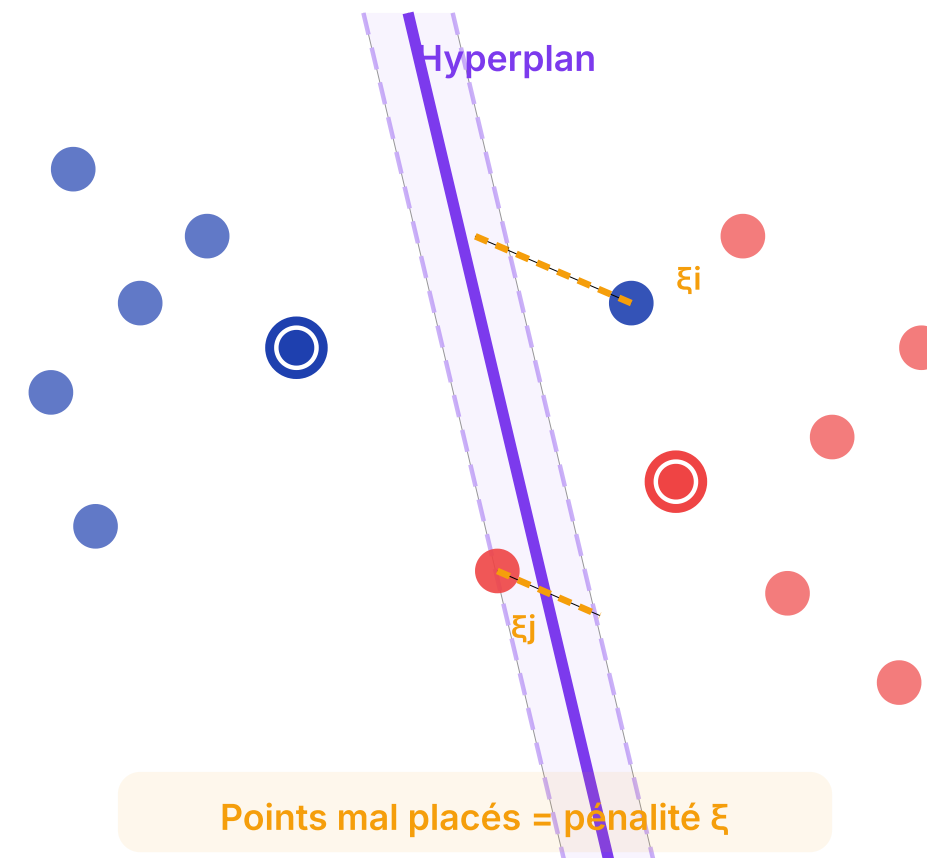
C petit

Très tolérant : plus d'erreurs, marge large

Variables de relâchement ξ_i

Chaque point mal classé ou dans la marge a un « coût » ξ_i proportionnel à sa distance du bon côté. Le SVM minimise les ξ_i tout en gardant la marge la plus large possible.

Marge poreuse (Soft Margin)



C est un hyperparamètre

Comme K dans KNN, **C ne s'apprend pas**. On le choisit par validation croisée. C trop grand = overfitting, C trop petit = underfitting.

Le Kernel Trick : séparer l'inséparable

N1 🦊 L'analogie du coup de poing

Des points rouges et bleus **mélangés sur une table**. Impossible de tracer une droite entre eux.

Donne un **coup de poing sous la table** : les points sautent en l'air à des hauteurs différentes. Dans cette 3e dimension, tu peux passer un plan entre les deux groupes !

💡 Le noyau fait cette projection **SANS** jamais calculer les nouvelles coordonnées.

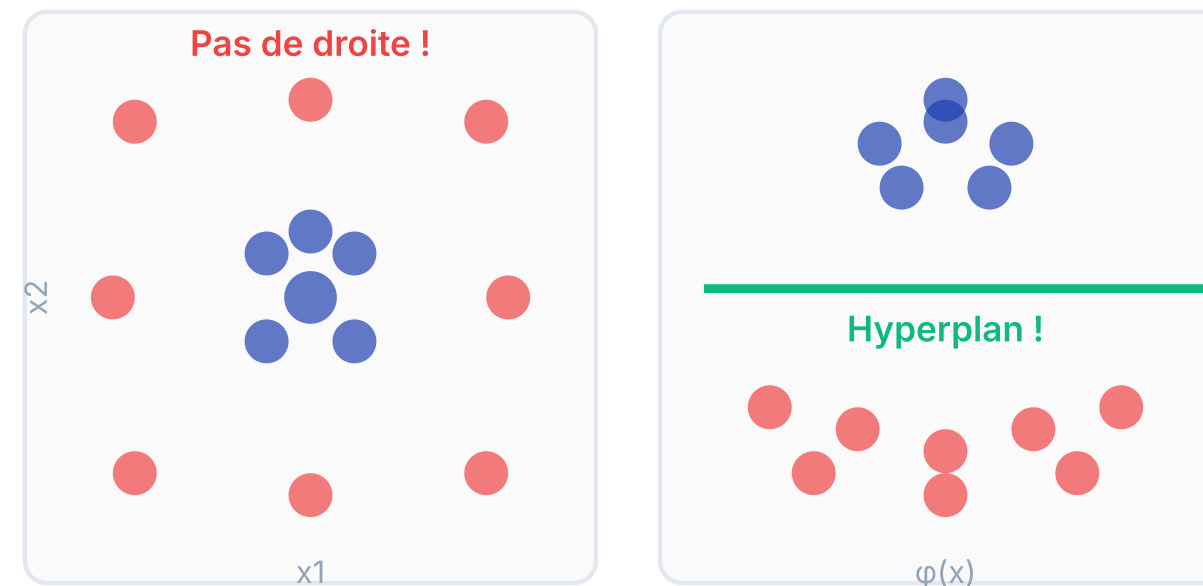
⚡ L'astuce du noyau

Au lieu de calculer $\phi(x)$ explicitement, on calcule directement $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$. C'est un raccourci mathématique qui évite de travailler dans l'espace de grande dimension.

Avant et après le Kernel Trick

Espace original (2D)

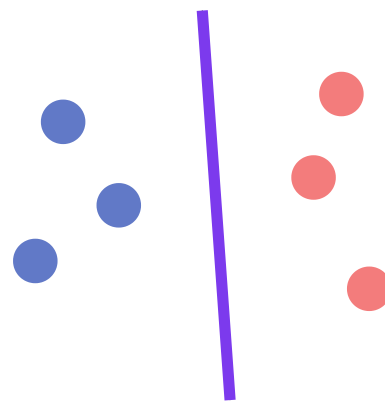
Après projection (3D)



👉 ϕ projette dans un espace où les données deviennent séparables

Les principaux noyaux

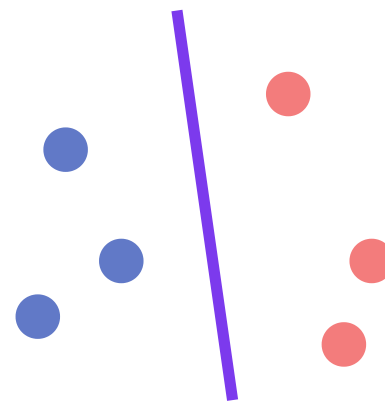
Linéaire



$$K = x \cdot y$$

Frontière droite

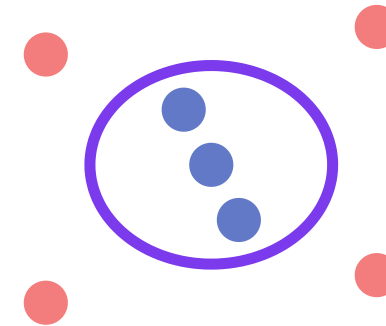
Polynomial



$$K = (x \cdot y + c)^d$$

Frontière courbe

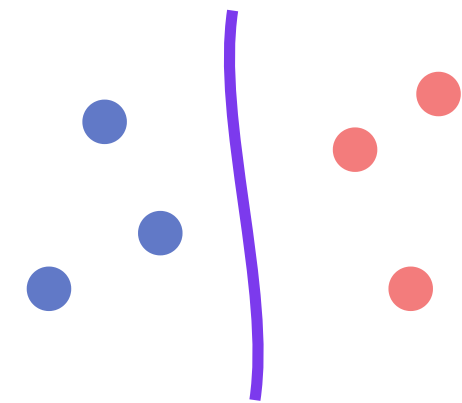
RBF (Gaussien)



$$K = e^{-\gamma \|x - y\|^2}$$

Premier choix en pratique

Sigmoïde



$$K = \tanh(\alpha x \cdot y + c)$$

Proche réseau neuronal

🎯 Règle pratique

Commencer par **RBF** (le plus flexible). Si les données sont linéairement séparables, le noyau linéaire suffit et sera plus rapide. Le paramètre γ du RBF contrôle le rayon d'influence de chaque point.

📖 Propriété fondamentale

On n'a **jamais besoin de calculer** la fonction de transformation ϕ . Le noyau $K(x_i, x_j)$ encode implicitement le produit scalaire dans l'espace transformé. C'est le « trick » !

Classifieur Bayésien : l'intuition

N1 ☁ L'exemple de la pluie en Tunisie

En Tunisie, il pleut **35 jours/an** en moyenne.

Ce matin, il y a des nuages. Quelle est la probabilité qu'il pleuve ?

On sait que :

- Nuages un jour de *pluie* : **9/10**
- Nuages un jour *sans pluie* : **3/10**

💡 **Les nuages sont 3x plus probables un jour de pluie !**

N2 Le raisonnement bayésien

1 Cote antérieure (a priori) : 35 : 330
Avant de regarder le ciel

2 Rapport de vraisemblance : $(9/10) / (3/10) = 3$
Nuages 3x plus probables si pluie

3 Cote postérieure (a posteriori) : $3 \times 35 : 330 = 105 : 330$
Après avoir vu les nuages

La règle de Bayes (version cote)

$$\text{cote postérieure} = \text{vraisemblance} \times \text{cote antérieure}$$



La règle de Bayes et l'hypothèse naïve

N2 Théorème de Bayes

$$P(C|X) = \frac{P(X|C) \times P(C)}{P(X)}$$

$P(C|X)$ = probabilité de la classe C sachant les données X (postérieure)

$P(X|C)$ = vraisemblance des données si la classe est C

$P(C)$ = probabilité a priori de la classe

$P(X)$ = constante de normalisation

N1 Pourquoi « naïf » ?

L'hypothèse **naïve** suppose que les features sont **indépendantes entre elles** sachant la classe.

Concrètement : si on classe des e-mails, on suppose que la présence du mot « million » n'influence pas la présence du mot « dollars ». C'est faux en réalité, mais **ça marche quand même bien** en pratique !

$$P(x_1, x_2, \dots, x_n|C) = \prod_{i=1}^n P(x_i|C)$$

Chaque feature contribue indépendamment

💡 Application médicale

Scanner détecte la maladie : **80/100**. Faux positif : **10/100**. Maladie touche **5%** de la population. Un test positif = quelle probabilité d'être malade ?

Rapport de vraisemblance : $80/10 = 8$. Cote antérieure : 5:95. Cote postérieure : $40:95 \approx 30\%$ (pas 80% !).

Application : filtre anti-spam

N2 Données d'entraînement

MOT	SPAM	NON-SPAM	VRAISEMBLANCE
million	156	98	5.1
dollars	29	119	0.78
adclick	51	0	∞
conference	0	12	≈ 0
Total mots	95 791	306 438	

Le rapport de vraisemblance = $\frac{P(\text{mot}|\text{spam})}{P(\text{mot}|\text{non-spam})}$
Un rapport > 1 : le mot penche vers **spam**.
Un rapport < 1 : le mot penche vers **non-spam**.

N3 Classifier « million dollars conference adclick »

- 1 Cote antérieure : 1:1 (50% spam, 50% non-spam)
- 2 Multiplier par chaque rapport de vraisemblance (hypothèse naïve : mots indépendants)

```
# Cote initiale
cote = 1

# Mot par mot (naïf = on multiplie)
cote *= 5.1    # million → penche spam
cote *= 0.78   # dollars → légèrement non-spam
cote *= 163.2  # adclick → très spam
cote *= 0.0003 # conference → très non-spam

# Résultat
cote_finale = 0.19 # < 1 → non-spam
```

Verdict

Cote finale ≈ 0.19 < 1 → **Non-spam**. Le mot « conference » a fait basculer la balance malgré « adclick ». C'est la puissance de Bayes : chaque preuve contribue.

Bayes Naïf : forces et faiblesses

Forces

- **Très rapide** à entraîner et à prédire (un seul passage sur les données)
- Fonctionne bien avec **peu de données**
- Excellente **baseline** (point de départ) pour tout problème de classification
- Naturellement **multi-classe**
- Insensible aux **features non pertinentes**

Faiblesses

- L'hypothèse d' **indépendance** est rarement vraie en pratique
- Les **probabilités estimées** ne sont pas fiables (trop extrêmes)
- Problème **zéro count** : si un mot n'apparaît jamais dans une classe, $P = 0$ → tout le produit = 0
- Ne capture pas les **interactions** entre features

Lissage

Ajouter une petite valeur (1/100000) pour éviter $P=0$

$O(n \times d)$

Entraînement linéaire en taille de données

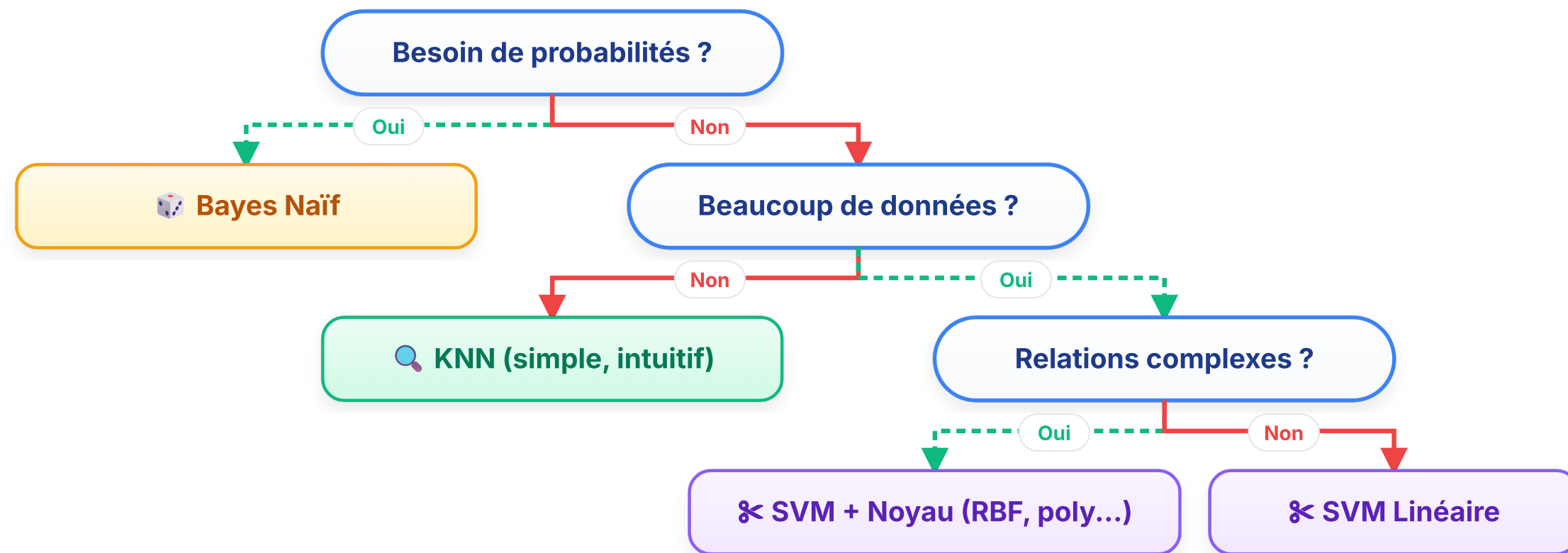
Texte

Domaine de prédilection : NLP, spam, sentiment

KNN vs SVM vs Bayes Naïf

CRITÈRE	KNN	SVM	BAYES NAÏF
Type	Non-paramétrique	Paramétrique	Probabiliste
Entraînement	Aucun (stocke tout)	Optimisation (trouver w, b)	Très rapide (compter)
Prédiction	Lente (compare à tout)	Rapide (calcul de f(x))	Très rapide
Gros datasets	Mauvais	Bon	Excellent
Haute dimension	Souffre (malédiction)	Excellent (noyaux)	Correct
Bruit / outliers	Sensible (K petit)	Robuste (soft margin)	Assez robuste
Interprétabilité	Intuitive (voisins)	Moyenne (vecteurs supports)	Bonne (probabilités)
Cas d'usage typique	Petits datasets, prototype rapide	Classification complexe, texte, images	NLP, spam, médical, baseline
Hyperparamètre clé	K (nb voisins)	C (tolérance), noyau, γ	Lissage

Quel algorithme choisir ?



Guide simplifié — en pratique, testez plusieurs modèles et validez (validation croisée).

Malédiction de la dimension : KNN souffre quand le nombre de features augmente (les distances perdent leur sens).

Validation croisée : diviser les données en K folds, entraîner sur K-1 et tester sur 1, répéter K fois.

Sur-apprentissage : un modèle qui apprend le bruit au lieu du signal. Se détecte quand l'erreur train est faible mais l'erreur test est forte.

— Messages clés

Ce qu'il faut retenir de ce chapitre



1. KNN : la force de la simplicité

Aucun entraînement, juste de la mémoire et de la distance. Intuitif pour débiter, mais inadapté aux gros datasets et à la haute dimension.



2. SVM : la puissance de la marge

Maximiser la marge = maximiser la généralisation. Le kernel trick ouvre la porte aux frontières non-linéaires sans exploser en complexité.



3. Bayes : penser en probabilités

La règle de Bayes met à jour nos croyances avec les preuves. L'hypothèse naïve est fausse mais étonnamment efficace en pratique.



4. Pas de solution universelle

Chaque algorithme a ses forces. Le bon choix dépend des données, de la taille, de la dimension et du besoin. Toujours comparer par validation croisée.